Property-based testing con QCheck

L09 · Programmazione funzionale · OCaml

Mauro Leoncini

Obiettivi

- Trasformare le proprietà di L08 in **test eseguibili**
- Capire che cos'è un **generatore** e perché fa parte della specifica
- Leggere un controesempio **ridotto al minimo**
- Le quattro famiglie di proprietà, e che cosa una suite verde **non** dice

La scala della verifica (da L02)

```
esempio          "su [3;1;2] restituisce 2"
   ↓
proprietà        "il risultato è un elemento della lista"
   ↓
controesempio    la macchina cerca l'ingresso che la rompe     (L09)
automatico
   ↓
dimostrazione    vale per OGNI ingresso, e lo so perché        (L08)
```

A L08 siamo saliti sull'ultimo gradino. Oggi torniamo al terzo ma non è un passo indietro: **controesempio e dimostrazione si controllano a vicenda**.

Una proprietà falsa si scopre in un secondo. Per dimostrare una proprietà vera ci vuole più tempo.

Prerequisiti

Per eseguire test su file singolo si può prescindere da `dune` (lo strumento per la creazione di progetti `OCaml` che automatizza molti passaggi). Gli step 2 e 3 sono quindi opzionali

1. Come prima cosa caricare le variabili d'ambiente di `opam`

- `eval $(opam env)`

2. Verificare che `dune` non sia già installato

- `dune --version`

3. Se non installato eseguire

- `opam install dune`

4. Quindi installare QCheck

- `opam install qcheck-core`

Anatomia di un test

```
open QCheck                                (* Import libreria *)
let map f l  = ...                         (* Funzioni coinvolte *)
let length l = ...                         (* nel test *)
Test.make                                  (* Chiamata di Test.make per
                                             fissare i parametri del test *)
  ~count:1000                              (* opzionale: quanti prove *)
  ~name:"length (map f l) = length l"      (* opzionale: nome del test *)
  ~print:Print.(list int)                  (* opzionale: come  stampare
                                             un controesempio*)
  Gen.(list nat_small)                        (* il GENERATORE *)
  (fun l -> length (map succ l) = length l) (* la PROPRIETÀ *)
let () = exit (QCheck_base_runner.run_tests_main [ invariant ] )
```

Lanciare il test

Due possibilità:

1. Con `dune`: nella directory che contiene anche file di test preparare i due file `dune-project` e `dune` (file inclusi nello `zip` di `L09` su moodle).

```
(* file dune; si suppone che il file di test si chiami prima.ml *)
 (test
 (name prima)
 (libraries qcheck-core qcheck-core.runner))
```

   poi lanciare il run `dune runtest`

2. Compilare e poi eseguire il file di test nel modo seguente

```
ocamlfind ocamlopt -package qcheck -linkpkg prima.ml -o first
./first
```

Esercizi per fare pratica

Associatività di append.

Enunciato Per ogni `xs`, `ys`, `zs`: `(xs @ ys) @ zs = xs @ (ys @ zs)`
Ricorrenza Sul primo argomento
Base `xs = []`. Dimostrazione immediata

```
      ([] @ ys) @ zs = ys @ zs          [A1]
      [] @ (ys @ zs) = ys @ zs          [A1]
```

Passo induttivo `xs = x :: r` e [II] `(r @ ys) @ zs = r @ (ys @ zs)`

```
      ((x :: r) @ ys) @ zs
    = (x :: (r @ ys)) @ zs              [A2] con ys
    = x :: ((r @ ys) @ zs)              [A2] con zs
    = x :: (r @ (ys @ zs))              [II]
    = (x :: r) @ (ys @ zs)              [A2] con (ys @ zs), da dx a sx
```

Esercizi per fare pratica

Invarianza della lunghezza rispetto al mapping.

Enunciato Per ogni `f`, `l`: `length (map f l) = length l`
Ricorrenza Sulla lunghezza della lista `l`
Base `l = []`.

```
    length (map f []) = length []    [M1]
```

Passo induttivo `l = x :: r` e [II] `length (map f r) = length r`

```
    length (map f (x :: r))
  = length (f x :: map f r)          [M2]
  = 1 + length (map f r)             [L2]
  = 1 + length r                     [II]
  = length (x :: r)                  [L2], da dx a sx
```

Esercizi per fare pratica

Proprietà di **fusione** del mapping.

Enunciato Per ogni `f, g, l`: `map f (map g l) = map (g %> f) l`
Ricorrenza Sulla lunghezza della lista `l`
Base `l = []`. Entrambe le parti valgono `[]`.

```
    map f (map g []) = map f [] = []          [M1, M1]
    map (g %> f) []    = []                   [M1]
```

Passo induttivo `l = x :: r` e [II] `map f (map g r) = map (g %> f) r`

```
    map f (map g (x :: r))
  = map f (g x :: map g r)                    [M2]
  = f (g x) :: map f (map g r)                [M2]
  = f (g x) :: map (g %> f) r                 [II]
  = (g %> f) x :: map (g %> f) r              [C], da dx a sx
  = map (g %> f) (x :: r)                     [M2], da dx a sx
```

Esercizi per fare pratica

Equivalenza delle due funzioni per **rovesciare** una lista.
Enunciato Per ogni `l`, `rev_acc [] l = naive_rev l`
Ricorrenza Sulla lunghezza della lista `l`
Base `l = []`. Entrambe le parti valgono `[]`.

```
    rev_acc [] [] = []          [RA1]
    naive_rev [] = []           [R1]
```

Passo induttivo `l = x :: r` e [II] `rev_acc [] r = naive_rev r`

```
    rev_acc [] (x :: r)
  = rev_acc [x] r               [RA2]
```

A questo punto, tuttavia, siamo costretti a fermarci perché l'ipotesi induttiva parla di `rev_acc [] r` mentre noi abbiamo `rev_acc [x] r`.

L'accumulatore è cambiato, e l'ipotesi induttiva (ma in realtà l'enunciato stesso) non dice nulla su accumulatori diversi da `[]`.

La soluzione sta nella generalizzazione

Per quanto la strada possa apparire più difficile, la soluzione consiste nel **generalizzare**, cioè dimostrare qualcosa di più forte

Enunciato Per ogni `acc`, `l`: `rev_acc acc l = naive_rev l @ acc`

Ricorrenza Sulla lunghezza della lista `l` e per un qualsiasi `acc`

Base `l = []`. Entrambe le parti valgono `acc`.

```
rev_acc acc [] = acc                [RA1]
naive_rev [] @ acc = [] @ acc       [R1]
                   = acc            [A1]
```

La soluzione sta nella generalizzazione (2)

Passo induttivo `l = x :: r` e
[II] per ogni `acc'`: `rev_acc acc' r = naive_rev r @ acc'`

Parte sinistra:

```
  rev_acc acc (x :: r)
= rev_acc (x :: acc) r                [RA2]
= naive_rev r @ (x :: acc)            [II, con acc' = x :: acc]
```

Parte destra:

```
  naive_rev (x :: r) @ acc
= (naive_rev r @ [x]) @ acc           [R2]
= naive_rev r @ ([x] @ acc)           [Associatività di @]
= naive_rev r @ (x :: ([] @ acc))     [A2]
= naive_rev r @ (x :: acc)            [A1]
```

Un semplice esercizio, un corollario e una riflessione

L'**esercizio**: dimostrare che `[]` è anche un'**identità destra** per `@`: `xs @ [] = []`

Il **corollario**: il teorema generalizzato insieme all'esercizio provano che:

```
  rev_acc [] l
= naive_rev l @ []            [Teorema 7 generalizzato, acc = []]
= naive_rev l                 [Esercizio]
```

che è il risultato che non eravamo riusciti a dimostrare.

La **riflessione**: L'enunciato generalizzato **non è suggerito dal tipo**. Dice: «ciò che è già nell'accumulatore, più il rovesciamento di ciò che resta, dà il risultato»: è l'**invariante del ciclo** di cui si parlava a L02.

L'accumulatore trasforma la ricorsione in un ciclo, e con il ciclo torna la difficoltà di inventarne l'invariante.

Quando una proprietà non vale

Ci siamo convinti che la **proprietà distributiva** di `map` rispetto a `@` valga invertendo le liste (magari ci siamo ricordati che `map` con `fold_left` inverte...)

```
map f (xs @ ys) = map f ys @ map f xs
```

Il test predisposto fallisce segnalando:

```
Test Distributive law: map f (xs @ ys) = map f ys @ map f xs failed
(XX shrink steps):

([0], [1])
```

Nota Il valore specifico XX può cambiare da run a run

Uno **shrink step** è un passo in cui QCheck cerca di trovare un controesempio più semplice partendo da uno un po' più complesso (rispetto a semplici criteri di valutazione)

Quattro famiglie

Quando non sapete che proprietà scrivere, provate queste, in quest'ordine:

andata e ritorno	`rev (rev l) = l`
invariante	`length (sort l) = length l`
oracolo di riferimento	`rev_acc [] l = naive_rev l`
metamorfica	`sort (rev l) = sort l`

Una funzione palesemente sbagliata

```
let fake_sort l = List.init (List.length l) (fun _ -> 0)
```

Restituisce una lista di zeri, lunga come l'ingresso.

- «il risultato è ordinato» — **passa**
- «la lunghezza non cambia» — **passa**

Che cosa mancava

```
List.sort compare (fake_sort l) = List.sort compare l
```

Fallisce, con il controesempio ridotto:

```
[1]
```

> Una suite verde non dice che il programma è giusto. Dice che le proprietà che avete scritto **non bastavano a distinguerlo** da uno sbagliato.

I generatori sono parte della specifica

Per le liste QCheck ha già tutto. Per un tipo vostro, il generatore lo scrivete voi:

```
let gen_tree =
  Gen.sized_size (Gen.int_bound 6) @@
  Gen.fix (fun self n -> match n with
    | 0 -> Gen.pure Leaf
    | n -> Gen.oneof_weighted
             [ 1, Gen.pure Leaf;
```

```
      3, Gen.map3 (fun l x r -> Node (l, x, r))
           (self (n/2)) Gen.nat_small (self (n/2)) ])
```

Il generatore sbagliato non si vede

Nello stesso file, una sola modifica: `Gen.int_bound 6` → `Gen.int_bound 1`.

```
success (ran 2 tests)
```

La proprietà **falsa** passa. Non è diventata vera: con quella taglia il generatore produce solo `Leaf` e `Node (L, x, L)`, e su quelli altezza e dimensione coincidono davvero.

Il referto dice `success` esattamente come quando è tutto a posto.

Che cosa portate a casa oggi

- Una proprietà è l'enunciato di L08, scritto in modo eseguibile
- Il controesempio **ridotto** vale molto più del primo trovato
- Quattro famiglie: andata e ritorno, invariante, oracolo, metamorfica
- Il **generatore** è parte della specifica: sceglie che cosa non verrà mai provato
- Verde non vuol dire corretto: vuol dire che le vostre proprietà non hanno saputo distinguerlo

Al calcolatore

```
lezioni/L09-qcheck/codice/
  prima.ml        anatomia di un test; togliete ~print e guardate
  leggi_L08.ml    le sei leggi dimostrate, e quella falsa
  famiglie.ml     le quattro famiglie, e la fake_sort che passa
  alberi.ml       il generatore scritto a mano; poi rompetelo
  esercizi.ml     quattro esercizi, di cui l'ultimo senza codice
  una serie di directory con i specifici test e i file `dune` e `dune-project`
```

```
ocamlfind ocamlopt -package qcheck -linkpkg prima.ml -o first && ./first
./prova -v          % mostra anche i test superati
./prova -s 12345    % fissa il seme, per ripetere una corsa
```